Platform Architecture for Onchain Operational Intelligence

Technical overview of the data, analytics, and application stack behind realtime monitoring, cross-protocol visibility, and risk analysis for complex digital-asset systems.

Roderick McKinley, CFA, FRM — Independent Financial Systems Analyst

The Architectural Premise

Purpose-Built Infrastructure for Operational Visibility

The platform is designed to convert raw onchain activity into queryable, decision-oriented monitoring across DEX, lending, yield, and issuer-controlled contract domains.

Rather than relying on protocol UIs or ad hoc scripts, it combines:

- Protocol-specific ingestion services
- In-database transformation and analytics
- Frontend monitoring applications
- Bounded recovery and health monitoring

The result is a structured monitoring layer built for active financial operations rather than passive observation.

End-to-End Architecture

From Onchain Data to Operational Views

Data Sources

Solana blockchain via Yellowstone gRPC, Solana RPC, and GraphQL indexer services

Ingestion Layer

Four protocol-specific Python services — DEXes, Exponent, Kamino Lending

Data Platform

TimescaleDB / Tiger Data with isolated schemas, raw tables, triggers, aggregates, and SQL view functions

Application Layer

HTMX UI service for dashboard delivery · Widget API + caching service for frontend-agnostic data serving

Operational Layer

Health endpoints, queue telemetry, reconnect / restart escalation, and backfill support

This architecture is designed as a complete data-to-decision pipeline, not just a collection of dashboards.

Ingestion Layer

Protocol-Specific Ingestion, Shared Platform Patterns

Each protocol domain is ingested by its own Python service, while following consistent shared architectural conventions.

Dual-channel ingestion

gRPC streams and RPC polling operating in parallel for each service.

Protocol-specific parsing

Transaction parsing and account decoding scoped to each protocol domain.

Write-on-difference controls

Unchanged state is suppressed to reduce load without sacrificing data freshness.

Isolated async write queues

Per data category — slow or failing writes in one category do not block others.

Dynamic account discovery

Monitored account sets evolve over time without service restarts.

Centralized environment-driven configuration

Per service — no bespoke one-off configuration patterns.

Why this matters: New protocol coverage can be added within a stable ingestion pattern rather than through bespoke one-off pipelines.

Data Engineering in the Database

In-Database ETL Built for Time-Series Analytics

The database is not just a storage layer. It is an active analytical layer.

Processing pipeline

01

Raw Ingestion

`src_*` source tables receive normalized raw records from ingestion services.

02

Enrichment

Triggers and auxiliary tables enrich and organize domain-specific data.

03

Aggregation

5-second continuous aggregates materialize time-bucketed rollups.

04

Serving

Parameterized SQL view functions serve frontend-ready results.

Design principles

Protocol schema isolation

Each protocol domain operates within its own schema boundary.

One source per concern

One source table per concern; one source per continuous aggregate.

Frontend reads from view functions

Not raw tables — SQL files remain the authoritative schema reference.

Why this matters: Protocol logic, time-series rollups, and reusable domain functions are composed close to the data.

Query Layer and Analytical Interface

Frontend-Facing Views, Not Raw Tables

The frontend and API layer consume the platform through parameterized SQL functions rather than direct access to raw tables.

This enables

Consistent query interfaces

Across pages and widgets — a single contract between the application and the data platform.

Flexible time windows and granularities

Re-bucketing of pre-aggregated 5-second data into frontend-appropriate intervals.

Clean separation of concerns

Ingestion, transformation, and presentation remain independently operable.

Examples of database-side logic

Price impact and liquidity traversal

AMM-specific traversal logic executed at query time against materialized aggregates.

Risk sensitivity calculations

Domain-specific analytics composed inside view functions and reused across the platform.

Protocol-specific AMM analytics

Domain logic encapsulated in the database, not duplicated in application code.

Application Layer

Server-Rendered UI + Widget API Architecture

The frontend is implemented as two coordinated FastAPI services.

HTMX UI service

Server-rendered Jinja pages

With HTMX widget updates and same-origin proxy routes.

Page orchestration

Page/module-based enablement and client-side interaction controls.

Same-origin proxying

Startup and runtime cache warmup with stale-while-revalidate caching.

Widget API + caching service

Frontend-agnostic JSON API

SQL access through a dedicated adapter layer — presentation layer is independent of data platform internals.

Cache policy and prewarm strategy

Pipeline-aware data serving with optional pipeline switching across configured environments.

Design intent: Presentation layer remains lightweight while preserving a reusable application boundary between UI and analytics.

Resilience and Recovery

Bounded Recovery, Health Monitoring, and Escalation

The platform follows a deliberate recovery philosophy: short-lived recovery inside the service; persistent failure escalates to container restart.

Three-tier recovery model

01

Internal reconnect

For transient gRPC and DB issues — bounded retry within the service.

02

Automatic thread restart

For failed subscription workers — isolated restart without full service teardown.

03

Platform restart

Via liveness failure or self-exit watchdog — escalation when internal recovery is exhausted.

Operational monitoring

Health endpoints on each service

Queue health and worker liveness checks alongside gRPC stream health monitoring.

Bounded recovery budgets

Services do not retry indefinitely — persistent failure escalates predictably.

Backfill workflows

Non-Kubernetes self-exit watchdog support and backfill workflows for recovering historical gaps.

Why this matters: The platform is designed to fail transparently and recover predictably, rather than silently degrading under partial failure.

Protocol and Domain Coverage

Built for Cross-Protocol Monitoring

The architecture is domain-isolated but operationally unified: protocol-specific ingestion and schema separation feed a shared analytical and monitoring layer.

Current monitored domains

- **Orca Whirlpool and Raydium CLMM**

  DEX liquidity and market structure across concentrated liquidity pools.

- **Exponent PT/YT markets**

  Yield and structured product exposure — vault state, rate dynamics, position behaviour.

- **Kamino Lending**

  Lending reserves and obligations — utilisation, liquidation conditions, and collateral concentration.

Coverage spans

- DEX liquidity and market structure
- Lending reserves and obligations
- Yield / structured products
- Primary market and issuer-controlled contract flows

External Dependencies and Platform Choices

Selected Infrastructure Partners

The platform relies on deliberately selected infrastructure components, each chosen to support specific technical requirements.

SHYFT

Realtime Solana data access

Yellowstone gRPC for realtime Solana streams · GraphQL indexer for large account-set discovery · RPC for polling and backfill support

Tiger Data

Analytical time-series database layer

PostgreSQL function extensibility · hypertables and continuous aggregates · complex joins and query-time engineering close to the data

Together, these infrastructure choices support:

- Low-latency ingestion
- Database-side analytical engineering
- Cross-domain time-series monitoring
- Extensibility without architectural drift

Design Principles

How the Platform Is Engineered

Schema as documentation

SQL definitions are heavily annotated and act as authoritative reference across the system.

Single source of truth per concern

Discriminators, config, schemas, and view logic each have defined homes — no ambiguity about where things live.

Queue isolation

Slow or failing writes in one category do not block others.

Write-on-difference

Unchanged state is suppressed to reduce load without sacrificing freshness.

Bounded recovery

Services do not retry indefinitely; persistent failure escalates predictably.

Platform-agnostic containers

Deployment model works across hosting environments with environment-level changes only.

These choices make the system easier to operate, extend, and reason about over time.

Close

Technical Credibility, Without Operational Sprawl

- The monitoring layer is structurally real
- The data flow is technically coherent end-to-end
- Analytics are engineered close to the data
- Resilience and recovery are first-class design concerns
- Protocol coverage can expand without rewriting the system

For teams evaluating monitoring, analytical infrastructure, or protocol-specific platform buildout, the architecture is designed to be understandable, extensible, and operationally serious.

Contact

Email: rmckinley@rmckinley.net
Website: https://rmckinley.net
LinkedIn: linkedin/in/roderickmckinley